

Runtime Enforcement in Stochastic Domains: A Formal Analysis of Shielded Reinforcement Learning

Luka Zaitch

Vrije Universiteit

September 26, 2026

Abstract

Reinforcement Learning (RL) has emerged as a powerful paradigm for solving complex decision-making problems, yet its fundamental reliance on stochastic exploration creates a critical barrier to deployment in safety-critical systems. This essay explores the concept of "Shielding" as a rigorous architectural solution to this conflict, effectively decoupling the learning objective from safety constraints through formal verification. We analyze the theoretical foundations of Shielding, defining the "Safety Game" played between the agent and an adversarial environment to derive "Winning Regions" of guaranteed safety. Furthermore, we contrast the implementation of Pre-Shielding versus Post-Shielding architectures and critically examine the limitations of absolute deterministic guarantees in stochastic domains. Finally, we discuss the development of Probabilistic (δ -bounded) Shields as a necessary relaxation to address the "paralysis problem," concluding with an analysis of the trade-off between model abstraction fidelity and computational tractability.

1 Introduction

Reinforcement Learning (RL) has demonstrated remarkable success in mastering complex tasks, from board games like AlphaGo to high-dimensional control problems. However, the standard RL paradigm is fundamentally built upon "trial-and-error" exploration: the agent must try new actions to discover their consequences. While acceptable in simulation, this approach is often prohibitive in safety-critical real-world domains such as autonomous driving, surgical robotics, or power grid management, where a single exploratory mistake can lead to catastrophic hardware damage or human injury [3].

The traditional method for encouraging safety in RL is "reward shaping"—assigning large negative rewards to unsafe states. However, this creates only a "soft constraint." The

⁰Collaboration Statement: This essay was authored by Luka Zaitch. Collaborators: None. Google Gemini was used in the literature review process and as a help in refining language use.

agent avoids failure not because it is impossible, but because it is numerically expensive. Crucially, to learn this penalty, the agent must often experience the failure state during training, which defeats the purpose of safety assurance. Furthermore, even a converged policy offers no rigorous guarantee that safety will be maintained in novel or stochastic situations [1].

To bridge the gap between high-performance learning and rigorous safety, we introduce the concept of the **Shield**. A Shield is a reactive runtime enforcement mechanism, synthesized via formal verification, that acts as a gatekeeper between the agent and the environment. By monitoring the agent’s actions and overriding those that violate a safety specification, the Shield enforces “hard constraints” that are correct-by-construction. This essay provides a technical analysis of Shielding, deriving the mathematical foundations of Safety Games and Winning Regions, and comparing the trade-offs between absolute deterministic safety and probabilistic (δ -bounded) risk models in the context of the “Sim-to-Real” abstraction gap.

2 Theoretical Fundamentals

To understand how Shielding guarantees safety in a stochastic Reinforcement Learning (RL) process, we must first formalize the environment and the safety specifications. The core innovation of Shielding is the translation of the safety problem into a 2-player game, formally referred to as a *Safety Game* [3].

2.1 The Environment: Markov Decision Processes (MDP)

Standard Reinforcement Learning assumes the agent interacts with an environment modeled as a Markov Decision Process (MDP). Formally, we define the MDP as a tuple $\mathcal{M} = (S, A, P, R, \gamma)$ [3]. In this framework, S represents the set of all possible states or configurations of the world, while A denotes the set of actions available to the agent.

The system’s evolution is governed by the transition probability function $P : S \times A \times S \rightarrow [0, 1]$, which acts as the fundamental law of motion for the environment. For any given state s and selected action a , $P(s'|s, a)$ defines a probability distribution over all possible next states s' . This function encodes the stochastic dynamics of the domain—representing factors like sensor noise, friction variability, or external disturbances—where a single action can lead to multiple different outcomes [3]. The critical safety challenge arises directly from these probabilistic mechanics: because the agent cannot deterministically predict the consequence of its actions, it is prone to triggering transitions to hazardous states during the necessary process of exploration [3].

To navigate this stochastic environment, the agent aims to learn a policy $\pi(s)$ that maximizes the expected cumulative reward, discounted by a factor $\gamma \in [0, 1)$. However, relying solely on the reward function $R : S \times A \rightarrow \mathbb{R}$ to enforce safety is fundamentally

flawed. In standard RL, safety violations are merely modeled as large negative rewards. This creates a "soft constraint" where the agent avoids crashes only because they are numerically expensive, not because they are strictly forbidden. Consequently, the agent is forced to experience these failures during training to learn the associated penalty, and even after convergence, it may still choose an unsafe action if the potential future reward outweighs the immediate risk [1].

2.2 Formal Specification: Linear Temporal Logic (LTL)

To overcome the limitations of soft reward constraints, we employ Linear Temporal Logic (LTL) to define "hard" safety specifications. To understand why this formalism is necessary, we must break down its components. The term "**Linear**" refers to the way we view the system's future: not as a branching tree of possibilities, but as a single, sequential timeline of states (a trace) extending forward. The "**Temporal**" aspect allows us to reason about the *ordering* of events along this timeline, rather than just the state of the system at a single snapshot.

This is critical because safety in continuous systems is rarely about a single instant; it is a property of the *entire infinite execution path*. For example, a robot is not "safe" just because it hasn't crashed *now*; it is safe only if we can guarantee it will not crash at any point in the indefinite future $(s_0, s_1, s_2, \dots)$. LTL allows us to capture this requirement mathematically. For safety shielding, we restrict our focus to the *safety fragment* of LTL, denoted as $G\phi$ ("Globally ϕ "), which asserts that a safety property ϕ must hold valid at every single time step in this infinite sequence [1].

However, an LTL formula describes a property of an abstract infinite path, which is difficult to verify instantaneously during a real-time process. To operationalize these specifications, we must translate the logical formula into a computational structure that can "run" alongside the agent: a *Deterministic Safety Automaton* (DSA). The DSA functions as a runtime monitor that converts the temporal logic into a state-based tracking mechanism. By maintaining its own internal state, the automaton effectively "remembers" the relevant history of the system, allowing it to verify the safety property incrementally at each time step.

Formally, this automaton is defined as a tuple $\mathcal{A}_\phi = (Q, \Sigma, \Delta, q_0, F)$, where Q is the set of automaton states and Σ is the alphabet of atomic propositions derived from the MDP. Crucially, the set $F \subseteq Q$ defines the "safe states." Alternatively, this is often conceptualized by the existence of a specific "sink state" or "error state" q_{err} . The DSA acts as a precise runtime monitor: it tracks the sequence of environment states and transitions to q_{err} if and only if the specification is violated. Once the system enters q_{err} , the safety property is irretrievably lost [3].

2.3 The Safety Game and Winning Regions

The central mathematical argument for Shielding relies on constructing a *Safety Game* by forming the product of the environment abstraction and the safety specification: $\mathcal{G} = \mathcal{M}_\phi \times \mathcal{A}_\phi$. This product structure encodes both the physical dynamics of the world and the logical progress of the safety monitor into a single state space.

The interaction is formally modeled as a contest between two opposing entities:

1. **The System (Shield):** Chooses actions to maintain the safety invariant.
2. **The Environment (Adversary):** Chooses transitions to force a violation.

Crucially, treating the environment as an "Adversary" is a necessary pessimism for verification. While the real physical world is not malicious, it is stochastic. To provide a rigorous safety guarantee, we cannot rely on "average" or "likely" outcomes; we must assume Murphy's Law. In this game, we assume that for any action the agent takes, the environment will resolve the probability distribution $P(s'|s, a)$ to the absolute worst possible next state s' . If the shield can win against this "evil" environment, it is guaranteed to remain safe under any random realization of the physical dynamics [3].

To guarantee safety, we must identify the *Winning Region* W . Formally, this is the set of all product states from which the Shield possesses a strategy to avoid q_{err} indefinitely:

$$W = \{(s, q) \in S \times Q \mid \exists \pi_{safe} \text{ s.t. } \forall \text{ paths from } (s, q), G(\text{Safe})\} \quad (1)$$

Intuitively, if the system is in a state $(s, q) \in W$, there exists at least one action that guarantees safety forever, regardless of how the environment behaves. This set is computed using a *backward reachability algorithm* that iteratively identifies states "doomed" to fail. Starting from the error states, the algorithm expands an "unsafe attractor"—the set of states from which the adversarial environment can force the system into a violation regardless of the shield's intervention. The Winning Region W is simply the complement of this attractor: the "safe harbor" of states from which the environment cannot force a failure [3].

2.4 Shield Synthesis

Once W is established, we synthesize the Shield σ as a reactive runtime component. For any current state s and automaton state q , the Shield computes the set of permissible safe actions A_{safe} by looking one step ahead:

$$A_{safe}(s, q) = \{a \in A \mid \forall s' \text{ with } P(s'|s, a) > 0, (s', \Delta(q, s')) \in W\} \quad (2)$$

This formula represents the "Correctness Guarantee." An action is deemed safe if and only if *every* possible future state resulting from that action resides within the Winning Region.

However, this strict requirement creates a fundamental limitation known as the **”Paralysis Problem.”** In stochastic environments, such as autonomous driving, theoretical worst-case scenarios (e.g., extremely rare sensor noise) often have non-zero probability. Because a deterministic shield forbids *any* action where the risk is greater than zero, it may block all useful behaviors, effectively freezing the agent to prevent a highly unlikely accident [3].

To address this, we consider **Probabilistic Shielding** as a necessary relaxation. Instead of requiring that all futures are safe, we require that the probability of landing in a safe state exceeds a high safety threshold. The shielding condition becomes:

$$\sum_{(s',q') \in W} P(s'|s,a) \geq 1 - \delta \quad (3)$$

Here, δ represents the bounded risk the system is willing to accept (e.g., $\delta = 0.01$), allowing the agent to trade absolute theoretical safety for the practical ability to operate in an uncertain world [2].

3 Shielding Architectures

Implementing a Shield requires integrating the formal verification logic—specifically the Winning Region W derived in Section 2.3—with the learning loop of the RL agent. This integration faces two primary challenges: the computational intractability of verifying high-dimensional continuous state spaces, and the specific mechanism by which the Shield intervenes in the agent’s decision-making process.

3.1 Constructing the Abstraction

Real-world environments, such as those encountered in robotics, often operate in continuous state spaces with infinite configurations. However, the synthesis of a safety strategy typically requires a finite, discrete model. To bridge this gap, we must construct an *abstraction* of the environment [1].

We define an abstraction mapping function $\phi : S \rightarrow S_\phi$, where S is the concrete state space and S_ϕ is a finite set of abstract states. The Shield operates entirely on the abstract level: it receives the concrete state s_t , maps it to $\hat{s}_t = \phi(s_t)$, and computes the safe strategy based on $\hat{s}_t$. Consequently, the safety guarantee is mathematically valid only with respect to the abstract model. This introduces the critical **”Reality Gap”**: if the abstraction $\mathcal{M}_\phi$ fails to capture the worst-case dynamics of the real environment (e.g., ignoring friction variability), the computed Winning Region may not actually guarantee safety in reality [3].

3.2 Pre-Shielding: Action Space Restriction

In the Pre-Shielding architecture, the Shield functions as a preemptive filter located between the environment and the learning agent. At every time step t , the Shield computes the set of theoretically safe actions $A_{safe}(s_t)$ derived from the Winning Region. Crucially, the Shield passes *only* this restricted subset $A_{safe} \subseteq A$ to the RL agent. The agent is then constrained to select an action a_t strictly from this safe subset.

Theoretically, this modifies the learning problem by making the action space dynamic and state-dependent. By effectively pruning the search space, the agent is prevented from ever "seeing" or exploring unsafe states. This creates a dense reward signal environment where the agent can focus purely on performance optimization without needing to learn safety constraints via trial-and-error. While this significantly accelerates convergence, it imposes a compatibility constraint: the underlying RL algorithm (e.g., Deep Q-Networks or Proximal Policy Optimization) must be capable of handling a variable number of available actions per state, a feature not universally supported in standard implementations [1].

3.3 Post-Shielding: Runtime Enforcement and Teaching

Alternatively, the Post-Shielding architecture positions the safety layer downstream from the agent's decision-making process, acting as a runtime monitor. In this setup, the RL agent operates on the full action space A and selects a tentative action a_t based on its current policy. The Shield intercepts this action and verifies it against the safe set $A_{safe}(s_t)$. If a_t is deemed safe, it is forwarded to the environment unchanged. However, if $a_t \notin A_{safe}(s_t)$, the Shield overrides the decision, substituting it with a safe action a'_t —typically chosen to minimize the deviation from the agent's original intent.

While this architecture is highly modular and compatible with any off-the-shelf RL algorithm, it introduces a significant "Teaching Problem." From the agent's perspective, it may select a dangerous action $a_{dangerous}$ but observe a safe transition and potentially a positive reward (resulting from the Shield's hidden correction). This disconnect can lead the agent to incorrectly value dangerous actions, creating a policy that relies entirely on the Shield for survival. To mitigate this, the Post-Shielding framework effectively modifies the environment's transition function P seen by the agent. Furthermore, to ensure the policy actually learns safety, the Shield must often inject an artificial negative reward (punishment) whenever an override occurs, explicitly signaling to the agent that its intended action was invalid [1, 3].

4 Deterministic vs. Probabilistic Guarantees

A critical distinction in the design of shielded RL systems is the nature of the safety guarantee provided. This choice fundamentally alters the behavior of the agent, particularly

in environments with inherent stochasticity.

4.1 Absolute Safety: The Conservative Approach

The standard shielding formulation, as described by Könighofer et al., operates on a premise of **Absolute Deterministic Safety**. In this framework, the environment is treated as an adversary in a zero-sum game. When calculating the Winning Region W , the synthesis algorithm assumes that if a transition to an error state is theoretically possible—no matter how unlikely—it will happen [3].

Mathematically, an action a is considered safe only if:

$$\min_{s' \in \text{supp}(P(s,a))} V_W(s') = 1 \quad (4)$$

where $V_W(s')$ is the indicator function that returns 1 if s' is in the Winning Region W and 0 otherwise. The term $\text{supp}(P(s,a))$ refers to the "support" of the distribution—the set of all future states s' that have a non-zero probability of occurring. This asserts a strict "weakest link" logic: if even a single future state lies outside the Winning Region, the minimum becomes 0. Thus, an action is accepted if and only if *every* possible outcome is guaranteed safe.

This worst-case analysis provides the strongest possible guarantee: the system will never crash. However, in stochastic domains (e.g., autonomous driving), this creates the "Paralysis Problem." If every useful action carries a microscopic probability of failure (e.g., a 0.001% chance of a sensor glitch), the support of the distribution will always contain an unsafe state. Consequently, a deterministic shield will identify *no* safe actions and halt the system entirely, often freezing the agent in its initial state to avoid even the slightest risk [3].

4.2 Probabilistic Safety: The δ -Shield

To resolve the paralysis caused by worst-case reasoning, we employ **Probabilistic Shielding**. As introduced in [2], this approach relaxes the binary notion of safety into a continuous quantity: the *probability* of avoiding unsafe states.

Instead of requiring absolute certainty, we introduce a risk threshold. A " δ -shield" permits an action if its safety probability is sufficiently close to the optimal safety probability achievable from that state. Formally, let $\text{val}_M(s, \alpha)$ be the probability of satisfying the safety specification from state s using action α , and let $\text{optval}_M(s)$ be the maximum possible safety probability from state s . The shielding condition becomes:

$$\text{val}_M(s, \alpha) \geq (1 - \delta) \cdot \text{optval}_M(s) \quad (5)$$

This formulation introduces a formal trade-off between safety strictness and exploration efficiency. By setting $\delta > 0$, we allow the agent to take actions that are slightly

suboptimal regarding safety but may offer significantly higher rewards. This prevents paralysis in scenarios where "perfect" safety is mathematically impossible, effectively filtering out *unnecessarily* dangerous actions while permitting *calculated* risks necessary for task completion [2].

5 Critical Analysis

While Shielding provides a rigorous mathematical framework for safe exploration, its practical application is constrained by two fundamental limitations: the fidelity of the environment model and the computational complexity of the synthesis process.

5.1 The Abstraction Gap: The "Sim-to-Real" Problem

The central premise of Shielding is that the safety guarantee is "correct-by-construction." However, this correctness holds *only* with respect to the abstract model $\mathcal{M}_\phi$, not necessarily the physical world. The process relies on an abstraction function ϕ that maps continuous, complex reality into a discrete state space.

If this abstraction $\mathcal{M}_\phi$ fails to capture the true dynamics of the environment—for instance, if it assumes a friction coefficient that differs from the real surface, or ignores temporal delays in sensor processing—the computed Winning Region W may essentially be a hallucination. In such cases, the Shield might permit an action it "thinks" is safe, leading to a catastrophic failure in reality. This disconnect is known as the "Sim-to-Real" gap or the "Abstraction Gap" [3]. Consequently, the guarantee provided by a Shield is not absolute physical safety, but rather *consistency with the model*.

5.2 Scalability vs. Fidelity Trade-off

To minimize the Abstraction Gap, one might intuitively increase the fidelity of the model (e.g., higher resolution grids, more complex physics). However, this triggers the "Curse of Dimensionality." The synthesis of the Safety Game (solving for W) generally requires iterating over the entire state space. As the model complexity increases linearly, the state space often grows exponentially.

This creates a fundamental conflict: **Scalability vs. Fidelity**.

- **High Fidelity:** Reduces the Abstraction Gap but makes the Safety Game computationally intractable due to state-space explosion.
- **High Scalability:** Requires coarser abstractions (simplifying the world), which makes verification fast but increases the risk that the model no longer represents reality accurately [2].

Current research attempts to mitigate this through techniques like "Iterative Weakening" or "Adaptive Shielding," but finding the optimal balance remains an open challenge in the field.

6 Conclusion

Reinforcement Learning fundamentally relies on exploration to discover optimal policies, a process that is inherently at odds with the strict safety requirements of critical systems. This essay has demonstrated that **Shielding** effectively resolves this conflict by enforcing a separation of concerns: the RL agent optimizes for performance, while a formally verified runtime monitor enforces safety as a hard constraint.

By treating the safety problem as a 2-player game between the system and an adversarial environment, Shielding transforms safety from a probabilistic outcome of training into a mathematical precondition of the system's operation. We have analyzed two primary architectural implementations—**Pre-Shielding** and **Post-Shielding**—and shown that while both guarantee safety relative to the model, they offer distinct trade-offs regarding learning efficiency and implementation complexity [1].

Furthermore, we critically examined the tension between theoretical rigor and practical utility. While deterministic shields provide absolute guarantees, they risk paralyzing the agent in stochastic domains. The introduction of **Probabilistic (δ) Shields** offers a necessary relaxation, allowing the system to accept bounded risks to maintain operational viability [2].

Ultimately, the efficacy of any shield is bound by the fidelity of its environment abstraction. The reliance on a known, accurate *a priori* model remains the primary bottleneck for widespread adoption. Consequently, the most promising direction for future research lies in "Learning the Shield"—developing methods to infer or refine safety automata directly from observational data, thereby closing the Sim-to-Real gap and extending formal guarantees to unknown environments [3].

References

- [1] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu. Safe reinforcement learning via shielding, 2017.
- [2] N. Jansen, B. Könighofer, S. Junges, A. C. Serban, and R. Bloem. Safe reinforcement learning via probabilistic shields, 2019.
- [3] B. Könighofer, R. Bloem, N. Jansen, S. Junges, and S. Pranger. Shields for safe reinforcement learning. *Communications of the ACM*, 68(11):80–90, 2025.